Lab 26 — Backup and Recovery with restic

> **SY0-701 Objective 3.4** — Explain the importance of resilience and recovery in security architecture
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~30 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: Take encrypted, deduplicated backups, simulate a ransomware attack, restore the clean snapshot while timing the recovery, replicate the repository off site, and schedule backups that meet a stated RPO.

You'll build: An encrypted restic repository with two snapshots, a verified restore, an off-site replica that passes an integrity check, and a cron schedule with a retention policy.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; no earlier lab needed; tools installed in step 1. The data and the repository password are synthetic placeholders.

Background

- The **3-2-1 rule**: three copies of the data, on two different media, with one copy **offsite**. Onsite copies restore fast; offsite copies survive fire, flood and site-wide ransomware.
- **Frequency** is driven by the **recovery point objective (RPO)** — how much data loss is acceptable. The restore method is driven by the **recovery time objective (RTO)** — how long recovery may take.
- **Encryption** of backups keeps a stolen backup from becoming a data breach. restic encrypts every repository with AES-256 and authenticates it with Poly1305.
- **Snapshots** capture a point in time; restic snapshots are deduplicated, so unchanged chunks are stored once. **Replication** copies the repository elsewhere, but replicates mistakes too — integrity checks and retention protect against that.
- **Recovery** must be tested: an untested backup is only a hope. Measure the restore time and verify the restored data.

Steps

1. Install restic and rsync

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y restic rsync
python3
restic version
```

2. Create production data and an encrypted repository

The repository password lives in a root-only file; the two environment variables point restic at the repository and password.

```
mkdir -p /data /backup /root/lab26
for i in $(seq 1 5); do echo "important record $i - synthetic customer data" >
/data/rec$i.txt; done
```

```
printf '%s\n' 'Lab26-Backup-Repo-Pass' > /root/lab26/restic.pass && chmod 600
/root/lab26/restic.pass
export RESTIC_REPOSITORY=/backup RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
restic init
```

3. Take a baseline snapshot and prove the repository is encrypted

```
export RESTIC_REPOSITORY=/backup RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
restic backup /data --tag baseline
restic snapshots
grep -r "important record" /backup >/dev/null && echo "PLAINTEXT FOUND" || echo
"Repository is encrypted - no plaintext on disk"
```

4. Change the data and take a second snapshot

Only the changed chunks are stored; `restic diff` shows exactly what changed between the two points in time.

```
export RESTIC_REPOSITORY=/backup RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
echo "updated balance - synthetic" >> /data/rec1.txt
echo "important record 6 - synthetic customer data" > /data/rec6.txt
restic backup /data --tag daily
read -r OLD NEW <<< "$(restic snapshots --json | python3 -c 'import sys,json;
s=json.load(sys.stdin); print(s[-2]["short_id"], s[-1]["short_id"])')"
restic diff "$OLD" "$NEW"
```

5. Simulate a ransomware attack

Overwrite and rename every file, and drop a ransom note — the production copy is now useless.

```
for f in /data/*.txt; do echo "ENCRYPTED_BY_RANSOMWARE" > "$f"; mv "$f" "$f.locked"; done
echo "Your files are encrypted. Pay 5 BTC to recover them. (lab simulation)" >
/data/README_RESTORE.txt
ls /data && cat /data/rec1.txt.locked
```

6. Restore the clean snapshot and time the recovery

Restore to a separate location first, verify it, quarantine the attacker's files as evidence, then put the clean data back. The `time` output is your measured RTO for this data set.

```
export RESTIC_REPOSITORY=/backup RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
time restic restore latest --target /restore
ls /restore/data && cat /restore/data/rec1.txt
mkdir -p /root/lab26/quarantine && mv /data/*.locked /data/README_RESTORE.txt
/root/lab26/quarantine/
cp -a /restore/data/. /data/
ls /data
```

7. Replicate the repository off site and verify integrity

The repository is already encrypted, so it can be copied to an untrusted "off-site" location as-is. `restic check` verifies the replica's structure and data.

```
export RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
mkdir -p /offsite
rsync -a /backup/ /offsite/
restic -r /offsite check --read-data
restic -r /offsite snapshots
```

8. Schedule backups and apply a retention policy

A four-hour schedule meets a four-hour RPO; nightly retention keeps hourly, daily and weekly points. Preview what retention would remove before relying on it.

```
cat > /etc/cron.d/restic-lab26 <<'EOF'
# Lab 26: back up /data every 4 hours (RPO = 4 h), then replicate off site
RESTIC_REPOSITORY=/backup
RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
0 */4 * * * root restic backup /data --tag scheduled --quiet && rsync -a /backup/
/offsite/
30 2 * * * root restic forget --keep-hourly 24 --keep-daily 7 --keep-weekly 4 --prune --
quiet
EOF
chmod 644 /etc/cron.d/restic-lab26 && cat /etc/cron.d/restic-lab26
export RESTIC_REPOSITORY=/backup RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
restic forget --keep-last 1 --dry-run
```

Test it

```
export RESTIC_PASSWORD_FILE=/root/lab26/restic.pass
cat /data/rec1.txt
ls /data/*.locked 2>/dev/null | wc -l
restic -r /offsite check && echo "Off-site replica verified"
```

Expected: `rec1.txt` shows the original record plus the `updated balance` line, `0` locked files remain in `/data`, and `no errors were found` followed by `Off-site replica verified`.

Troubleshooting

Symptom	Fix
`Fatal: Please specify repository location (-r or --repository-file)`	You opened a new terminal; re-run the `export RESTIC_REPOSITORY=... RESTIC_PASSWORD_FILE=...` line.
`Fatal: create repository at /backup failed: ... config file already exists`	The repository was initialised in an earlier run; skip `restic init` and continue.
`restic diff` fails with `ambiguous` or empty IDs	List IDs with `restic snapshots` and pass the two short IDs to `restic diff` by hand.

Challenge

Make the off-site copy ransomware-resistant: run a local MinIO server with object locking (or use `chattr +i` on the replica files as a simple stand-in) and show that a root process can no longer modify the replicated

repository. Then explain why immutable storage and separate backup credentials matter for ransomware.

Reflection

The business says it can lose at most one hour of orders and must be trading again within two hours. Using your measured restore time and the cron schedule, does this design meet the RPO and RTO? What would you change?

Exam tips

- RPO = maximum tolerable data loss (sets backup frequency); RTO = maximum tolerable downtime (sets the restore method).
- Full, incremental and differential differ in what each run copies and what a restore needs; snapshots are point-in-time and often on the same storage.
- Onsite backups are fast; offsite backups survive site disasters; encrypt both and test restores regularly.
- Replication and journaling improve recovery, but replication copies corruption — it is not a substitute for backups.

References

- restic documentation — https://restic.readthedocs.io/
- NIST SP 800-34 Rev. 1 Contingency Planning Guide — https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final
- CISA #StopRansomware Guide — https://www.cisa.gov/stopransomware/ransomware-guide
- CIS Critical Security Controls v8 (Control 11 Data Recovery) — https://www.cisecurity.org/controls

---